

RFP Sentinel

Evaluation Pipeline Architecture — Phase 1 & Phase 2 (Stage 1 + Stage 2)

Rule-Based Scoring & Compliance-Reliability Redesign

Technical documentation prepared for project evaluation

Covers the current, deployed architecture of the RFP processing phase and the bid evaluation phase,
including all redesign work completed this development cycle.

1. Purpose & Scope

This document describes the current architecture of RFP Sentinel's two-part evaluation pipeline, with emphasis on the redesign work completed in the most recent development cycle:

- **Phase 1 — RFP Processing:** turns an uploaded RFP PDF into structured, checkable criteria, flags any that conflict with government procurement norms or GeM's own prohibited-practice rules, and pauses for human review (Checkpoint A) before publishing.
- **Phase 2 — Bid Evaluation (Stage 1 + Stage 2):** once bids are submitted and the RFP closes, Stage 1 checks each bid's evidence against the approved criteria and computes a technical score; Stage 2 ranks technically-qualified bids by price (L1) or a price/quality blend (QCBS), pausing for final human confirmation (Checkpoint B).

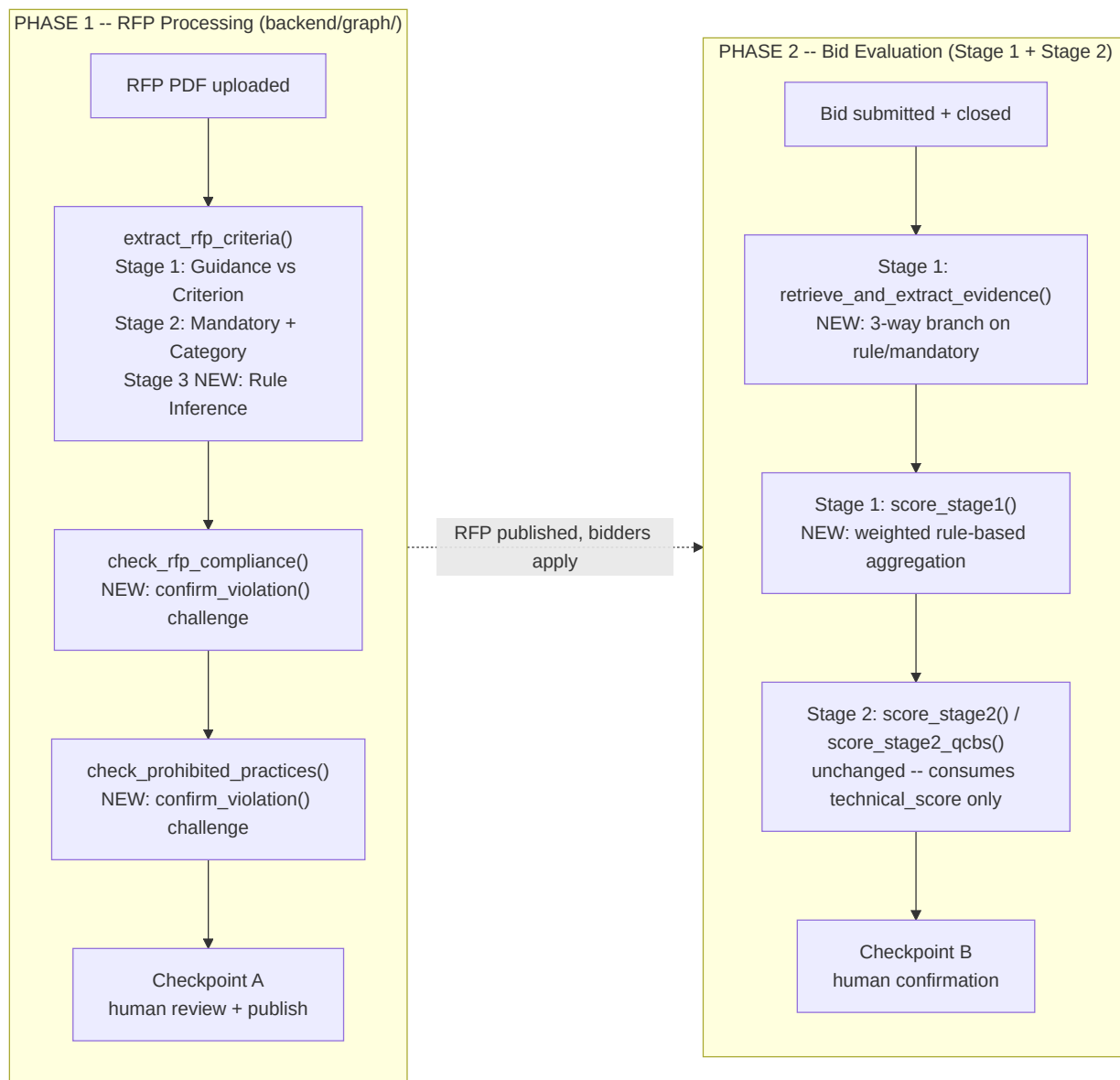
Everything in this document reflects the **actual, current, tested state of the codebase** — every architectural claim, code path, and test result cited here was directly verified against the running system, not assumed. Sections are marked **NEW** where this cycle changed behavior, and **UNCHANGED** where prior, already-proven logic was deliberately left untouched.

1.1 What Changed, in One Paragraph

Two things were redesigned this cycle. First, the **scoring engine** moved from a crude system where every scored criterion counted equally (a plain average of pass/fail/partial verdicts) to a **rule-based engine**: an LLM extracts each criterion's own stated marks/points breakdown (if the RFP states one) into a structured, typed rule; a small, pure, deterministic function evaluates that rule against a bidder's evidence; and the final technical score is a properly weighted sum, not an equal-weighted average. Second, the **reliability of the compliance/prohibited-practice checks** was hardened after repeatedly finding real false positives across live-tested RFPs — through a full prompt redesign (generalized reasoning instructions replacing hardcoded exception lists), a second adversarial verification pass before any flag reaches a buyer, and the discovery and fix of a silent context-window truncation bug that had been cutting prompts off mid-content.

2. Overall Architecture

The diagram below shows both phases end to end. Every box marked with an inline note is part of this cycle's redesign; everything else is prior, unchanged infrastructure.



Key structural fact worth stating explicitly

Stage 2 (backend/scoring/scoring.py — `score_stage2()` / `score_stage2_qcbs()`) required **zero changes** in this redesign. It only ever consumes a bid's already-computed `technical_score` (a single float), price, MSE/MII status — it never looks at individual criteria. This was confirmed by direct re-reading of the module before any redesign work began, and it means the entire scoring-engine redesign is architecturally contained to Stage 1: evidence extraction and per-criterion scoring

aggregation. The "ranking layer never needs to know what a criterion measures" principle was already true; the redesign only had to make Stage 1 honor it properly.

3. Phase 1 — RFP Processing

3.1 Criteria Extraction — `backend/graph/extract_rfp_criteria.py`

For every candidate clause chunked out of the uploaded PDF, three concurrent stages run across a bounded thread pool (`_MAX_CONCURRENT_LLM_CALLS = 5`):

- **Stage 1 — Guidance vs. Criterion:** is this clause a real bidder-facing requirement, or buyer-facing boilerplate (disclaimers, drafting instructions)? `UNCHANGED mechanism` , prompt content redesigned this cycle (see §3.3).
- **Stage 2 — Mandatory + Category:** is this a hard gate or a scored/preferential criterion? Which of technical / financial / eligibility / other does it belong to?
- **Stage 3 — Rule Inference** `NEW` : for criteria in a *scored* category (technical, financial, other — eligibility is gate-only and skipped), a two-step LLM check decides whether the RFP states an explicit marks/points breakdown for this criterion, and if so, extracts it into a structured `Rule` .

Rule Inference in detail

Module: `backend/graph/extract_scoring_rule.py` , function `infer_rule(criterion_text) -> Rule | None` :

1. **Gate** — a cheap `classify()` call: `has_rule` vs `no_rule` . Only answers `has_rule` if actual numbers are tied to specific tiers/options in the text itself.
2. **Shape extraction** — only if gated in: an `extract_json()` call asks the model to produce one of two typed shapes. The result is validated by attempting to parse it as a `Rule` ; any validation failure (or an empty response) degrades to `rule=None` , never raises — the same "don't guess" discipline used throughout this codebase.

The Rule schema — `backend/models/rule.py` `NEW`

```
RangeRule { type: "range", field: str, tiers: [{min_value, max_value, score}, ...] }
LookupRule { type: "lookup", field: str, table: {"": score, ...}, default_score }
```

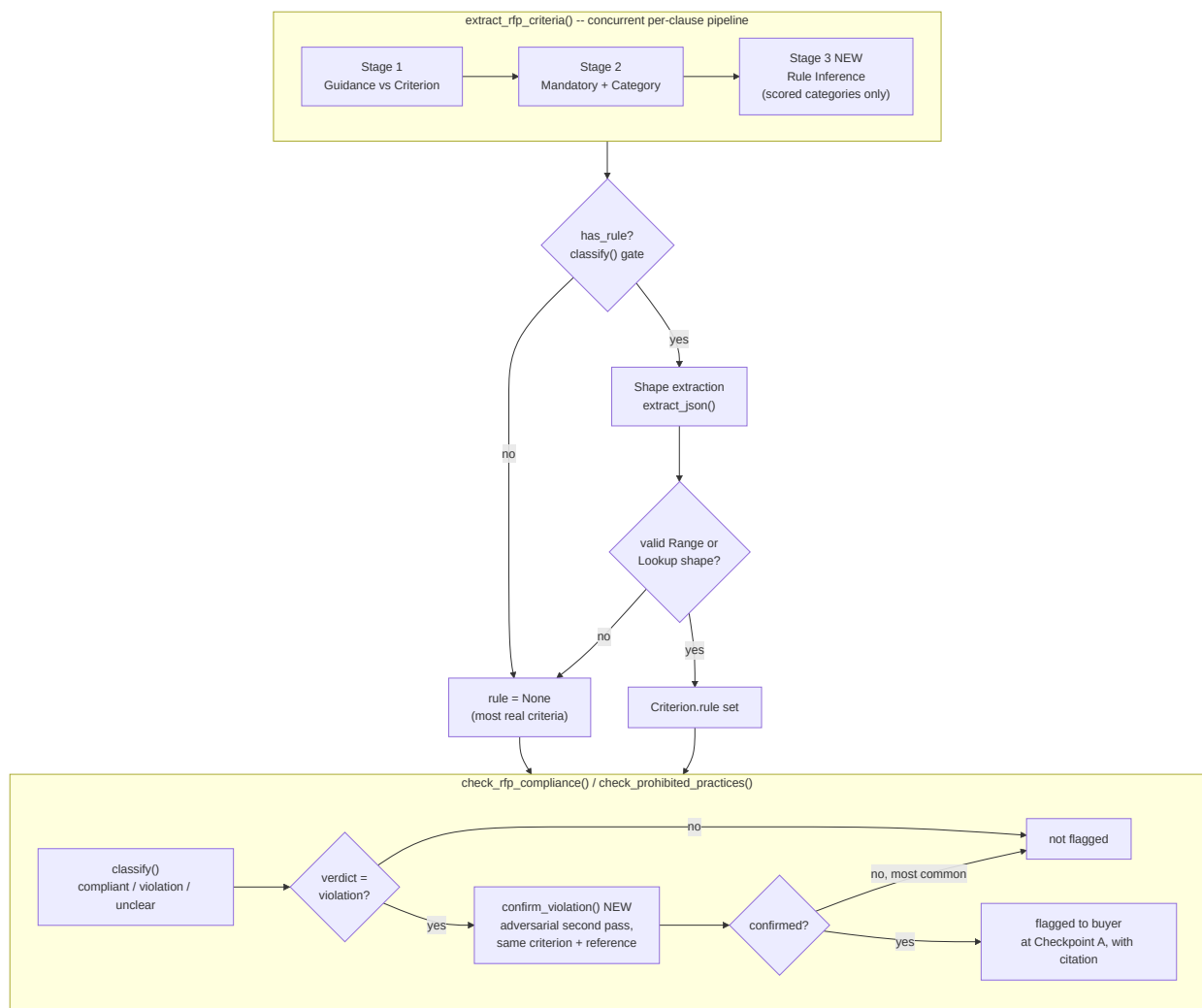
A binary/boolean condition (e.g. "has ISO 27001 certification: +10 marks") is expressed as a two-entry `LookupRule` (`{"true": 10, "false": 0}`) rather than a third rule type — one fewer schema shape, one fewer evaluator branch, for identical capability. `max_score` is never stored directly; it is always *derived* (max of a range's tier scores, or a lookup table's values), so the model extracting a rule can never state a number inconsistent with its own tiers.

`Criterion` (`backend/models/rfp.py`) gained exactly one new field: `rule: Rule | None = None` . It defaults to `None` , so any RFP checkpointed before this field existed deserializes unchanged — a strict superset of the old model, not a breaking change. `mandatory` and `category` are unchanged, orthogonal axes: `mandatory` still decides the hard eligibility gate; `rule` only changes *how* a scored criterion's contribution to the technical score is computed.

A formula-based rule type ("revenue ratio → marks") was deliberately **not** built. Formulas of that kind typically need a reference value from *outside* the single bid being scored (e.g. the highest bidder's revenue) — something this evaluator, by design, never has access to. Revisit only if a real RFP is found with a formula that is genuinely self-contained per bid.

3.2 Compliance & Prohibited-Practice Checking

Modules: `backend/graph/check_rfp_compliance.py` (criterion vs. retrieved government norm clauses) and `backend/graph/check_prohibited_practices.py` (criterion vs. this RFP's own extracted list of GeM's buyer drafting-mistake prohibitions). Both follow the identical pattern shown in the diagram below.



The challenge step — `confirm_violation()` NEW

Function in `backend/llm/ollama_client.py` . Whenever the first classification returns `violation` with a citation, a second, independent, adversarial call re-examines the *same* criterion and reference from scratch — explicitly instructed to be skeptical of the first pass's own conclusion, and specifically to catch the pattern "the criterion merely restates/aligns with the reference" being mistaken for "the criterion contradicts it." Only a `confirmed` result from this second pass is ever surfaced to a buyer; `not_confirmed` silently downgrades the flag. This doubles LLM cost only for the risky verdict (a small minority of checks), not for every criterion.

Why this exists: real false positives were found live, more than once, on unrelated real RFPs, all sharing the same failure shape — a criterion that quotes or restates a norm's own rule (e.g. a Conflict-of-Interest clause almost verbatim identical to GeM's own GTC clause) being flagged as a "violation" of that same rule. Prompt-wording fixes alone reduced this but did not eliminate it (see §6).

3.3 The Shared LLM Reasoning Layer

REDESIGNED

`classify()` and `extract_json()` (`backend/llm/ollama_client.py`) are the two functions every LLM call in this pipeline goes through. This cycle's redesign established one convention for every caller: an `instruction` string must be *only* a ROLE + OBJECTIVE + DECISION RULES block. The harness itself supplies the reasoning/grounding instruction and the output-format contract exactly once, so it never has to be re-explained (or re-invented, inconsistently) at each of the eleven call sites across this codebase.

Before	After
Hardcoded exception lists baked into individual prompts — e.g. naming specific brands ("Dell"), specific government schemes (DPIIT, MSME) as things <i>not</i> to confuse with a violation.	General reasoning rules — e.g. "classify 'violation' only if the criterion actually names or points to a specific supplier... not because it states a required attribute in generic terms." No proper nouns anywhere.
Grounding/output instructions repeated, inconsistently worded, at every call site.	Stated once in the shared harness; task instructions only state what's unique to that task.

A more elaborate, explicit multi-step "reasoning process" scaffold was tried in the shared harness and **reverted** after live A/B testing showed it made the model *more* prone to false "violation" flags, not less — instructing the model to "identify the specific decisive fact" before reading references appeared to anchor it on the topic level rather than the specific claim, giving it more room to construct a plausible-sounding connection instead of verifying one. The single-line "think through your reasoning, then answer" version in production today empirically produced fewer false positives on the same real-RFP test set while still catching the same genuine violations. This is documented directly in the module's own docstring as a lesson for future prompt changes.

3.4 The Context-Window Fix

NEW

A significant, independently-discovered bug: Ollama runs a model with `num_ctx` defaulting to roughly 2048 tokens *regardless of what the model itself supports*, unless a caller explicitly requests more. Confirmed empirically — a real 24,000-character prompt (a criterion plus five retrieved norm chunks)

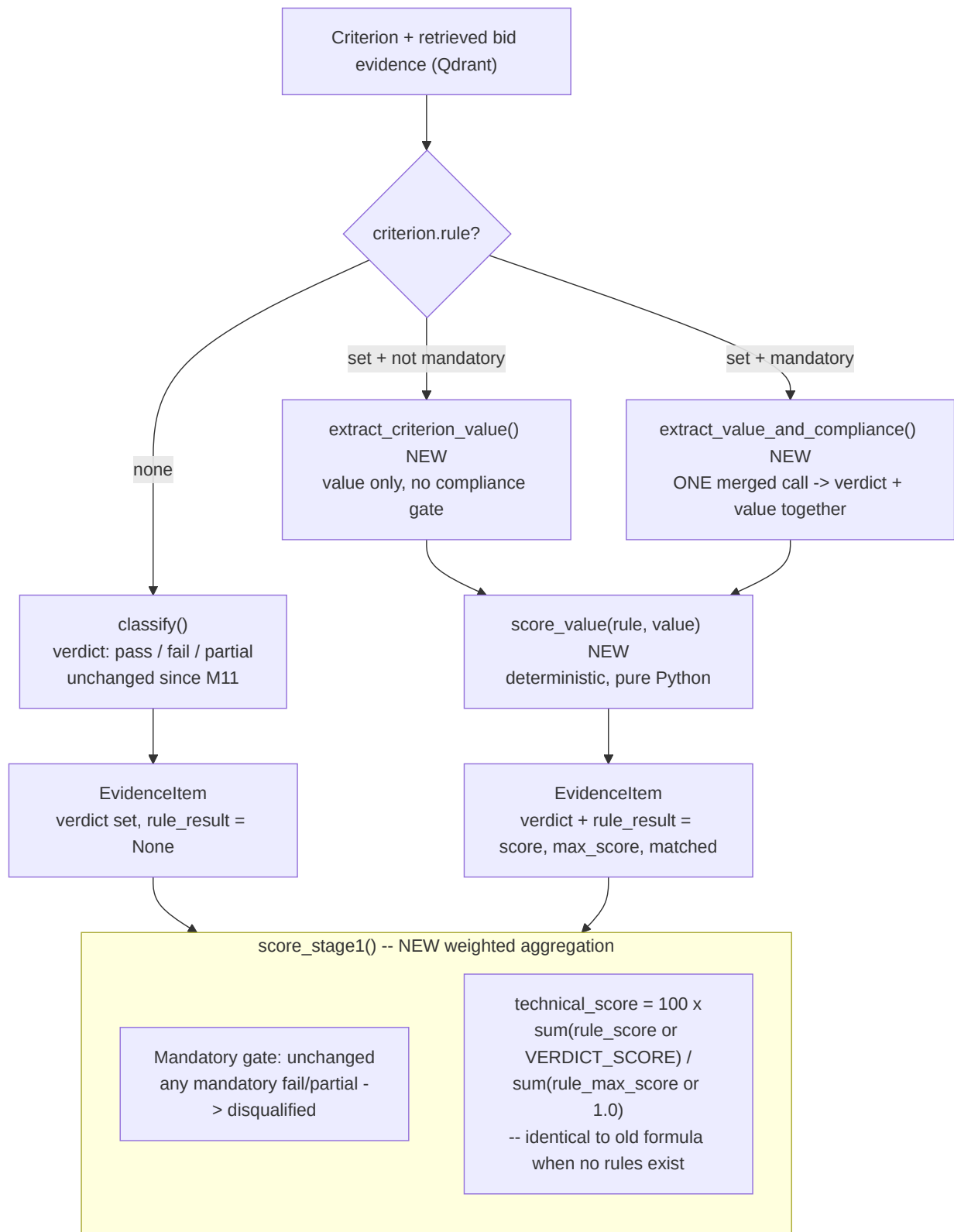
and a deliberately oversized 35,000-character test prompt both reported the identical `prompt_eval_count: 2050`, and the oversized prompt's own trailing instruction was silently dropped and never answered by the model.

Every prompt in this pipeline is built as `instruction → references → subject text → output-format instructions`, in that order. A ~2048-token ceiling risked silently truncating away the actual criterion text and the output-format instructions on any prompt with substantial reference material — meaning the model could be reasoning from an incomplete prompt regardless of how well it was worded. Fix: `num_ctx` is now explicitly set to 8192 in every request (`backend/llm/ollama_client.py`), comfortably covering every real prompt measured in this pipeline and well within the model's native 32K context.

4. Phase 2 — Bid Evaluation (Stage 1 + Stage 2)

4.1 Stage 1 — Evidence Extraction REDESIGNED

Module: `backend/graph/retrieve_and_extract_evidence.py` . Compliance (the mandatory gate) and scoring (marks earned) are treated as independent concepts, resolved via one of three paths per criterion — never two independent LLM calls interpreting the same evidence separately, which was found to risk disagreeing with each other.



- **rule is None** (the common case): today's original `classify()` flow, byte-for-byte unchanged. `verdict` drives the mandatory gate as always.
- **rule set, not mandatory**: no gate applies, so only a value is needed — `extract_criterion_value()` asks for exactly the rule's field as a single JSON value, grounded against the same retrieved bid chunks.
- **rule set, mandatory** (uncommon, e.g. "5+ years mandatory, more years = more marks"): **one merged call**, `extract_value_and_compliance()`, answering both the compliance verdict and

the rule's value from a single reading of the same evidence — so the two answers can't independently disagree.

`EvidenceItem` (`backend/models/evidence.py`) gained one new field: `rule_result: dict | None` (an `{score, max_score, matched}` blob). `verdict` keeps its existing meaning for the mandatory gate; for a non-mandatory rule-scored criterion it is only a coarse presence indicator, never a compliance judgment.

4.2 The Deterministic Rule Evaluator NEW

Module: `backend/scoring/criterion_evaluator.py`, function `score_value(rule, value) -> RuleResult`. Pure Python, zero LLM/Qdrant imports — same discipline, and same style, as `scoring.py` next to it: a plain function, not a class (this codebase has no "Engine" class pattern anywhere it didn't already have one). Two dispatch branches:

- **Range**: walks tiers in declaration order, first tier whose `[min_value, max_value]` contains the extracted value wins. A value that matches no tier (a gap, or below/above every tier) scores 0 with a `not_found` match — same signal as no value at all.
- **Lookup**: normalizes the extracted string (lowercase, strip punctuation — the same normalization already used elsewhere in this codebase for prohibited-practice matching) and looks it up in the table; an unmatched value falls back to `default_score`.

Critically, per direct design-review correction during this cycle: the evaluator returns **only** a score, never a pass/fail/partial verdict. A bidder landing in a lower-marks tier has still *fully* satisfied that tier — treating a lower tier as "partial compliance" was an early design mistake, caught and corrected before shipping (a hand-written test, `test_range_mid_tier_is_not_partial_credit`, exists specifically to guard against this regressing).

4.3 `score_stage1()` Weighted Aggregation REDESIGNED

Module: `backend/scoring/scoring.py`. The mandatory gate loop is completely untouched — still keyed purely on `evidence.verdict`. Only the scored-criteria aggregation changed:

```
raw_sum = max_sum = 0.0
for c in scored_criteria:
    e = evidence_by_criterion.get(c.id)
    if e and e.rule_result is not None:
        raw_sum += e.rule_result["score"]; max_sum +=
e.rule_result["max_score"]
    else:
        verdict = e.verdict if e else "not_found"
        raw_sum += VERDICT_SCORE[verdict] * 1.0; max_sum += 1.0    # old
implicit weight
technical_score = (raw_sum / max_sum * 100) if max_sum > 0 else 0.0
```

Backward compatibility is provable, not just asserted: when zero criteria have a rule, `max_sum == len(scored_criteria)` exactly, making this formula algebraically identical to the old plain average.

Every pre-existing unit test in `tests/test_scoring.py` passes unmodified against the new code — the regression proof that this is a strict superset of the old behavior, not a replacement with different edge cases.

4.4 Stage 2 — Ranking UNCHANGED

`backend/scoring/scoring.py` — `score_stage2()` (L1: MII filter → price rank → MSE price-match band) and `score_stage2_qcbs()` (price + technical-score blend, default 70/30 weighting). Neither function was touched. Both already only consume `BidInput` (price, MII/MSE flags) and a plain `technical_scores: dict[bid_id, float]` — the exact "ranking engine never knows what a criterion measures" property this whole redesign was aiming for already existed here.

5. Why This Redesign Was Needed

5.1 The scoring gap

The original engine's `technical_score` was a flat average of categorical verdicts — every scored criterion was worth exactly the same $1/N$ share of the total, regardless of what the RFP itself said it was worth. A real ATC section stating "Experience: 20 marks, GPU: 25 marks, Support: 15 marks" had no way to be honored; the engine would have scored all three equally. This is the gap the Rule schema and weighted aggregation close.

5.2 The false-positive investigation

Across repeated live testing against real, distinct RFP documents this cycle, the compliance/prohibited-practice checks produced confirmed false positives on several unrelated real clauses — an RFP clause deferring to "the applicable GeM GTC" being flagged as violating that same GTC; a criterion restating GeM's own Conflict-of-Interest / related-bidder rule almost verbatim being flagged as contradicting it; a generic technical specification (RAM, power supply count) initially at risk of being confused with a prohibited brand-naming practice. Every one of these shares the same underlying failure shape: mistaking "this criterion shares a topic with, or restates, the reference" for "this criterion contradicts the reference." Three independent mitigations were layered against this (prompt redesign, the challenge step, the context-window fix), each closing a different slice of the problem — see §6 for exactly which case each one fixed.

6. Verification Evidence

Every row below is a real, live test result from this development cycle — not a projection.

Test case	Before	After	What fixed it
ATC clause deferring to "applicable GeM GTC"	violation	compliant	Prompt redesign (self-deferral decision rule)
ATC clause citing a specific GTC sub-clause number	violation	compliant	Prompt redesign
Generic RAM/power-supply spec vs. brand-naming prohibition	(new pattern, untested)	compliant , correctly reasoned via the general "generic spec ≠ named source" rule	Prompt redesign generalized correctly to unseen phrasing
Conflict-of-Interest clause restating GTC almost verbatim	violation	violation (1st pass) → correctly downgraded	Challenge step (confirm_violation)
Conflict-of-Interest disjunctive-list clause ("...; or")	violation , survived even the challenge step (which itself misread the clause)	unclear , correct, first pass	Context-window fix — this case needed the full prompt, not another wording iteration
Synthetic true positive (5% discount offered vs. 10% mandatory minimum)	n/a	violation , correctly confirmed	Proves the mitigations don't over-correct into never flagging anything real
Full Stage 1/2 synthetic run (range, lookup, boolean-lookup, mandatory+rule, non-mandatory+rule, no-rule, 2 bids)	n/a	Every aggregation number matched hand-calculated math exactly (e.g. $68/75 = 90.67\%$)	Confirms the deterministic engine has no arithmetic bugs

All 49 unit tests in `tests/` pass throughout every change described in this document — none of this redesign touched a function signature, a Pydantic schema, or an API contract.

7. Known Limitations & Open Items

Stated plainly, matching this project's standing discipline of tracking what's proven versus what's assumed:

- **Rule extraction is architecturally complete but empirically unproven on real graded-scoring content.** Across six distinct real RFPs tested this cycle, zero criteria ever had a rule extracted — every one of these documents uses the L1 (lowest-price-wins) evaluation method, where eligibility is pass/fail and no explicit marks table is typically stated. The synthetic Stage 1/2 test (§6) proves the mechanism is correct once a rule exists; it has not yet been exercised against a real QCBS-style RFP with an actual stated marks table.
- **The challenge step measurably helps but is not proven bulletproof.** It has a directly observed failure case of its own (confabulating a reading of a disjunctive-list clause) — later resolved by the unrelated context-window fix, not by the challenge step itself. Treat it as a real risk reduction, not a guarantee.
- **A separate, still-open extraction-quality issue:** certain multi-column tables (e.g. consignee/delivery schedules) are not being detected as tables by the current PDF library, causing scrambled, garbled text to appear as a nonsensical "criterion." A direct, untuned comparison test showed PyMuPDF's table detector reconstructing the exact same table cleanly with zero document-specific configuration — a promising lead for a future, properly-scoped migration, not yet implemented.